

RecyclerView e ActivityResult no Android

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

Até agora, usamos a classe `ListView` para exibir a lista de contatos do nosso projeto **AgendaApp**. Revisando rapidamente, para implementar um `ListView` com layout personalizado em Android, utilizamos um `ArrayAdapter` customizado que permite inflar um layout específico para cada item da lista. O processo envolve alguns passos fundamentais:

1. Criar a *classe de modelo* (por exemplo, `Contato.kt`), que representará cada item da lista;
2. Criar o *layout XML do item da lista* (como `item_contato.xml`), contendo os componentes visuais desejados, como `ImageView` para foto e `TextView` para nome, telefone e email;
3. Implementar uma *classe adaptadora personalizada*, estendendo `ArrayAdapter<Contato>`, sobrescrevendo o método `getView()` — nesse método, é essencial utilizar o parâmetro `convertView` para *reaproveitar views recicladas* e evitar inflar novamente o layout sempre que possível, melhorando a performance.
4. Instanciar o adaptador na `Activity` e associá-lo ao `ListView` usando `listView.adapter = ContatoAdapter(...)`.

Por meio de um `ArrayAdapter` com um layout customizado, conseguimos apresentar a foto, nome e telefone de cada contato. No entanto, conforme o app cresce e se torna mais complexo, começamos a perceber **limitações importantes** da abordagem com `ListView`:

- **Reutilização ineficiente de views:** Apesar do uso do `convertView`, a reutilização de views ainda depende de cuidados manuais, o que pode gerar bugs e perda de desempenho.
- **Layout fixo:** O `ListView` só suporta **layouts verticais**. Se quisermos uma disposição em grade (grid), ou horizontal, precisamos trocar de componente.

- **Desempenho ruim com listas grandes:** O `ListView` não foi projetado para lidar com grandes volumes de dados com eficiência.

Para sanar este gargalo, em 2014 foi introduzido o `RecyclerView` e atualmente é recomendado seu uso pela própria [documentação](#): “Também recomendamos usar `RecyclerView` em vez de `ListView`, já que ela pode reciclar os layouts de itens individuais da lista, o que é mais eficiente e pode melhorar a performance de rolagem.”

O que é o RecyclerView?

O `RecyclerView` é um componente do Android projetado para exibir listas (ou grades) de itens com **alto desempenho, modularidade e extensibilidade**. Assim como o `ListView`, ele serve como um “repetidor” de elementos, mas foi pensado para atender às necessidades modernas de aplicativos, como animações, múltiplos tipos de layout, e atualização de dados com fluidez. O `RecyclerView` veio como evolução natural ao `ListView`, oferecendo uma estrutura mais **modular, performática e flexível**.

Estrutura do RecyclerView

A arquitetura do `RecyclerView` é composta por **componentes separados e bem definidos**, que trabalham em conjunto:

Componente	Função
<code>RecyclerView</code>	O container visual onde os itens são exibidos.
<code>Adapter</code>	Responsável por criar os itens (ViewHolders) e ligar os dados .
<code>ViewHolder</code>	Representa cada item da lista com seu próprio layout.
<code>LayoutManager</code>	Define como os itens são dispostos : em coluna, linha ou grade.
<code>ItemDecoration</code>	Permite adicionar divisórias, margens e outros elementos visuais.
<code>ItemAnimator</code>	Controla animações de inserção, remoção e mudança de itens.

Implementando o RecyclerView

1) Alterar o layout da MainActivity

Substituir `ListView` por `RecyclerView` no `activity_main.xml`.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:background="#F9F9F9"
    android:padding="16dp"
    tools:context=".ui.MainActivity">

    <TextView
        android:id="@+id/tvTitulo"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:text="Agenda de Contatos"
        android:textSize="36sp"
        android:textStyle="bold"
        android:textColor="#333"
        android:gravity="center"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintHeight_percent="0.15"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:id="@+id/btnAdicionar"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Adicionar Contato"
        android:textAllCaps="false"
        android:backgroundTint="@android:color/holo_green_light"
        android:textColor="#FFFFFF"
        android:textSize="18sp"
        android:elevation="4dp"
        android:layout_marginTop="16dp"
        app:layout_constraintHeight_percent="0.1"
        app:layout_constraintTop_toBottomOf="@id/tvTitulo"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/listViewContatos"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:divider="@android:color/holo_purple"
        android:dividerHeight="2dp"
        android:background="@android:color/white"
        android:elevation="10dp"
        android:padding="8dp"
        android:listSelector="@android:color/holo_green_light"
        app:layout_constraintTop_toBottomOf="@id/btnAdicionar"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHeight_percent="0.7"
        app:layout_constraintStart_toStartOf="parent">
```

```

        app:layout_constraintEnd_toEndOf="parent" />
    </androidx.constraintlayout.widget.ConstraintLayout>

```

2) Refatorar ContatoAdapter e criar o ViewHolder com ViewBinding

```

class ContatoAdapter(
    private val contatos: List<Contato>,
    private val onClick: (Contato) -> Unit
) : RecyclerView.Adapter<ContatoAdapter.ViewHolder>() {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val binding = ItemContatoBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        holder.bind(contatos[position])
    }

    override fun getItemCount(): Int = contatos.size

    inner class ViewHolder(val binding: ItemContatoBinding) : RecyclerView.ViewHolder(binding.root) {
        fun bind(contato: Contato) {
            binding.imgFoto.setImageResource(contato.foto)
            binding.tvNome.text = contato.nome
            binding.tvTelefone.text = contato.telefone
            binding.tvEmail.text = contato.email
            binding.root.setOnClickListener { onClick(contato) }
        }
    }
}

```

A classe `ContatoAdapter` é responsável por conectar os dados da lista de contatos (`List<Contato>`) com os itens visuais exibidos dentro do `RecyclerView`. Diferente do `ArrayAdapter`, que possui uma estrutura simplificada e genérica, o `RecyclerView.Adapter` exige uma abordagem mais modular, baseada na criação de um `ViewHolder` para representar cada item da lista. Ao estender `RecyclerView.Adapter`, somos obrigados a implementar três métodos fundamentais:

- `onCreateViewHolder`: é chamado quando o Android precisa **criar uma nova View** para um item da lista. Aqui, utilizamos o ViewBinding para inflar o layout `item_contato.xml`, tornando o código mais limpo e seguro.
- `onBindViewHolder`: é chamado para **preencher os dados do item na posição indicada**. Dentro do método `bind(contato: Contato)`, associamos os valores das propriedades do objeto `Contato` às views do layout, e também adicionamos o `setOnClickListener` ao `binding.root`, que permite capturar o clique em todo

o item da lista. A função `onClick`, passada como parâmetro na criação do adaptador, delega à `Activity` a definição do que acontecerá quando um contato for clicado — como, por exemplo, abrir a tela de detalhes daquele contato.

- `getItemCount`: informa ao `RecyclerView` quantos itens há na lista, ou seja, o tamanho da lista de contatos.

No interior do `ContatoAdapter`, usamos uma **inner class** chamada `ViewHolder`, que estende `RecyclerView.ViewHolder`. Em Kotlin, uma **inner class** é uma classe interna que mantém uma referência implícita à classe externa. Isso permite que o `ViewHolder` acesse diretamente membros da `ContatoAdapter` caso necessário, como a lista de contatos ou o callback `onClick`. Essa abordagem é comum em adapters porque facilita o acesso e a organização do código. A inner class `ViewHolder` recebe um parâmetro `binding: ItemContatoBinding`, que é um objeto gerado pelo View Binding com base no layout `item_contato.xml`. Esse binding é obtido no método `onCreateViewHolder` através de `ItemContatoBinding.inflate(...)` e já vem com todas as referências aos componentes do layout, como `imgFoto`, `tvNome`, etc., evitando a necessidade de usar `findViewById`.

3) Atualizar MainActivity

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var contatos: MutableList<Contato>
    private lateinit var adapter: ContatoAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        loadData()
        setupRecyclerView()
        setupListeners()
    }

    private fun loadData() {
        contatos = mutableListOf(
            Contato(R.drawable.minkey, "Minkey", "(16) 98888-7777", "minkey@email.com"),
            Contato(R.drawable.mennie, "Mennie", "(16) 97777-6666", "mennie@email.com"),
            Contato(R.drawable.patetla, "Patetla", "(16) 96666-5555", "patetla@email.com"),
            Contato(R.drawable.truco, "Truco", "(16) 98888-4444", "truco@email.com"),
            Contato(R.drawable.donaldo, "Donaldo", "(16) 95555-2222", "donaldo@email.com"),
            Contato(R.drawable.carlos, "Carlos", "(16) 94444-1111", "carlos@email.com")
        ).sortedBy { it.nome }.toMutableList()
    }
}
```

```

private fun setupRecyclerView() {
    adapter = ContatoAdapter(contatos) { contato ->
        val intent = Intent(this, DetalheContatoActivity::class.java)
        intent.putExtra("contato", contato)
        startActivity(intent)
    }

    binding.listViewContatos.layoutManager = LinearLayoutManager(this)
    binding.listViewContatos.addItemDecoration(
        DividerItemDecoration(this, DividerItemDecoration.VERTICAL)
    )
    binding.listViewContatos.adapter = adapter
}

private fun setupListeners() {
    binding.btnAddicionar.setOnClickListener {
        val intent = Intent(this, CadastroContatoActivity::class.java)
        startActivity(intent)
    }
}
}

```

A `MainActivity` foi reorganizada para se adequar à nova arquitetura com `RecyclerView`, que substitui o antigo `ListView`. A principal mudança é a introdução do método `setupRecyclerView()`, responsável por configurar o comportamento da lista de contatos na interface. O `ContatoAdapter` é criado e recebe como parâmetro a lista de contatos e uma *lambda* que define o comportamento quando o usuário clica em um item. Neste caso, abrimos uma nova `Activity` de detalhes com os dados do contato selecionado. O `LinearLayoutManager` é atribuído ao `RecyclerView` para que os itens sejam dispostos em coluna vertical. Esse gerenciador é o equivalente funcional ao comportamento padrão do `ListView`, mas com mais controle e flexibilidade. Existem três tipos principais: o `LinearLayoutManager`, que posiciona os itens em uma única linha (vertical ou horizontal) e é o mais comum em listas simples; o `GridLayoutManager`, que organiza os elementos em formato de grade, ideal para visualizações em blocos, como catálogos de imagens ou produtos; e o `StaggeredGridLayoutManager`, que também forma uma grade, mas permite que os itens tenham alturas ou larguras diferentes, criando um efeito visual em cascata semelhante ao do Pinterest. O `DividerItemDecoration` adiciona linhas divisórias verticais entre os itens, simulando o comportamento visual típico do `ListView`.

Uma prática comum no Kotlin é agrupar várias operações sobre um mesmo objeto com a função de escopo `apply`, que permite evitar repetição do nome do objeto. Quando

percebemos que há muitas chamadas sequenciais em um mesmo objeto, como por exemplo:

```
binding.listViewContatos.layoutManager = LinearLayoutManager(this)
binding.listViewContatos.addItemDecoration(
    DividerItemDecoration(this, DividerItemDecoration.VERTICAL)
)
binding.listViewContatos.adapter = adapter
```

Pode ser reescrita como:

```
binding.listViewContatos.apply {
    layoutManager = LinearLayoutManager(this@MainActivity)
    addItemDecoration(DividerItemDecoration(context, DividerItemDecoration.VERTICAL))
    adapter = this@MainActivity.adapter
}
```

Em Kotlin, o `this` é uma palavra-chave que se refere à instância atual de uma classe. Porém, quando você está dentro de funções *lambda* (como as que usamos no `apply`, `let`, `run`, etc.), o `this` pode mudar de significado, e **passa a apontar para o contexto da *lambda*** e não mais para a *Activity*. `this@MainActivity` é uma forma de especificar de maneira clara o contexto da *Activity* atual (`MainActivity`), especialmente quando estamos dentro de um escopo onde o `this` se refere a outro objeto. Por exemplo, ao utilizar funções como `apply`, o `this` dentro do bloco passa a apontar para o objeto sobre o qual o `apply` está agindo — no caso, um `RecyclerView`. Quando precisamos passar o *context* da *Activity* para criar um `LinearLayoutManager`, por exemplo, usamos `this@MainActivity` para garantir que estamos nos referindo à instância da *Activity*, e não ao `RecyclerView`. Essa técnica é útil para evitar ambiguidade e torna o código mais explícito e legível.

ActivityResult: Retornando dados com Intent

Em aulas anteriores, aprendemos a abrir a `DetalheContatoActivity` passando um **objeto Contato** por meio de uma *Intent*. Esse fluxo funcionou perfeitamente, pois a tela de detalhes só precisava **exibir** informações. No nosso projeto `AgendaApp`, até agora trabalhamos com o envio de informações entre *Activities* de forma unidirecional. Por exemplo:

- A `MainActivity` envia um objeto `Contato` para a `DetalheContatoActivity` via `Intent`.
- A `DetalheContatoActivity` apenas **exibe** os dados e, ao final, chama `finish()`.

Agora, queremos **criar um novo contato** a partir de um formulário (`CadastroContatoActivity`) e **retornar** esse novo contato para a tela principal (`MainActivity`), que deverá exibir o novo item na lista. Porém, ao criar a tela de **cadastro de contatos**, surge uma nova necessidade: Queremos permitir que o usuário **preencha um formulário e retorne esse novo contato para a tela principal**, para que ele seja imediatamente adicionado à lista.

A lógica parece simples:

1. Abre a tela `CadastroContatoActivity`;
2. Usuário preenche os campos;
3. Ao clicar em “Salvar”, o contato é criado e enviado de volta para a `MainActivity`.

Para adicionarmos esta funcionalidade, precisaremos de usar `ActivityResult`, que é exatamente uma `Activity` que retorna algo através de uma `Intent`. Por padrão, quando somente chamamos `startActivity(intent)`, o sistema operacional não cria um canal de comunicação para `Intents`. Usar uma `ActivityResult` nos possibilita criarmos uma `Intent`, colocar dados nela e enviar para a `Activity` originária.

Vamos refatorar a classe `MainActivity`:

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var contatos: MutableList<Contato>
    private lateinit var adapter: ContatoAdapter
    private lateinit var launcherCadastro: ActivityResultLauncher<Intent>

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        loadData()
        setupRecyclerView()
        setupLauncherCadastro()
        setupListeners()
    }
}
```



```

private fun loadData() {
    contatos = mutableListOf(
        Contato(R.drawable.minkey, "Minkey", "(16) 98888-7777", "minkey@email.com"),
        Contato(R.drawable.mennie, "Mennie", "(16) 97777-6666", "mennie@email.com"),
        Contato(R.drawable.patetla, "Patetla", "(16) 96666-5555", "patetla@email.com"),
        Contato(R.drawable.truco, "Truco", "(16) 98888-4444", "truco@email.com"),
        Contato(R.drawable.donaldo, "Donaldo", "(16) 95555-2222", "donaldo@email.com"),
        Contato(R.drawable.carlos, "Carlos", "(16) 94444-1111", "carlos@email.com")
    ).sortedBy { it.nome }.toMutableList()
}

private fun setupRecyclerView() {
    adapter = ContatoAdapter(contatos) { contato ->
        val intent = Intent(this, DetalheContatoActivity::class.java)
        intent.putExtra("contato", contato)
        startActivity(intent)
    }

    binding.listViewContatos.apply {
        layoutManager = LinearLayoutManager(this@MainActivity)
        adapter = this@MainActivity.adapter
        addItemDecoration(
            DividerItemDecoration(context, DividerItemDecoration.VERTICAL)
        )
    }
}

private fun setupLauncherCadastro() {
    launcherCadastro = registerForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { result ->
        if (result.resultCode == Activity.RESULT_OK) {
            val data = result.data
            val novoContato = data?.getSerializableExtra("contato", Contato::class.java)

            if (novoContato != null) {
                contatos.add(novoContato)
                adapter.notifyItemInserted(contatos.size - 1)
            }
        }
    }
}

private fun setupListeners() {
    binding.btnAddicionar.setOnClickListener {
        val intent = Intent(this, CadastroContatoActivity::class.java)
        launcherCadastro.launch(intent)
    }
}
}

```

Aqui, refatoramos a `MainActivity` com o uso do `ActivityResultLauncher`, uma API moderna e recomendada para substituir o antigo `startActivityForResult` (hoje considerado obsoleto). Na função `setupLauncherCadastro()`, criamos esse launcher com `registerForActivityResult(...)`, especificando o contrato `StartActivityForResult()`. A lambda passada como callback define o que será feito quando a `CadastroContatoActivity` retornar: se o resultado for `RESULT_OK`, extraímos

o `Contato` serializado da `Intent` retornada e adicionamos à lista com `contatos.add(...)`, notificando o `adapter` com `notifyItemInserted(...)`, que irá atualizar nosso `RecyclerView`.

Agora será necessário refatorar a classe `CadastroContatoActivity`:

```
class CadastroContatoActivity : AppCompatActivity() {

    private lateinit var binding: ActivityCadastroBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityCadastroBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupListeners()
    }

    private fun setupListeners() {
        binding.btnSalvar.setOnClickListener {
            val nome = binding.edtNome.text.toString()
            val telefone = binding.edtTelefone.text.toString()
            val email = binding.edtEmail.text.toString()

            if (nome.isNotBlank() && telefone.isNotBlank() && email.isNotBlank()) {
                val novoContato = Contato(
                    foto = R.drawable.ic_launcher_foreground,
                    nome = nome,
                    telefone = telefone,
                    email = email
                )

                val resultIntent = Intent().apply {
                    putExtra("contato", novoContato)
                }

                setResult(Activity.RESULT_OK, resultIntent)
                finish()
            } else {
                Toast.makeText(this, "Preencha todos os campos", Toast.LENGTH_SHORT).show()
            }
        }

        binding.btnVoltar.setOnClickListener {
            setResult(Activity.RESULT_CANCELED)
            finish()
        }
    }
}
```

Já na `CadastroContatoActivity`, quando o usuário preenche o formulário e clica em “Salvar”, criamos um novo objeto `Contato` com os dados digitados. Em seguida, preenchemos uma `Intent` com esse contato via `putExtra(...)`, e usamos `setResult(...)` para sinalizar que a operação foi bem-sucedida (`RESULT_OK`). Com isso, o `ActivityResultLauncher` da tela anterior é acionado automaticamente com esse retorno. Se os campos estiverem vazios, exibimos um `Toast` de aviso. Se o usuário desistir, clicando em “Voltar”, usamos `setResult(RESULT_CANCELED)` para indicar o cancelamento à `MainActivity`, para que ela saiba que não será retornado nada.

Agora, nosso aplicativo consegue adicionar um novo item ao nosso `RecyclerView`, porém ainda percebe que não conseguimos definir uma foto, nem persistir os dados (ao ser fechado, os dados adicionados se perdem) que é o que veremos na próxima aula.

Exercício de Fixação – Cadastro de Serviços

Implemente a refatoração aplicada ao AgendaApp em seu aplicativo de listagem de serviços usados como projeto avaliativo.